# Action Directives — User Guide

**Revision:** 1 **Last modified:** 2026-07-14T21:50:00Z **Authority:** constitution §11.4.140 (action-prefix system) + §11.4.202 (reporting directives) **Maintainer:** constitution submodule **Scope:** for anyone *using* the directives. If you are wiring them into a project, read ADMIN_MANUAL.md instead.

Companion documents: REFERENCE.md · QUICKSTART_REPORT.md · QUICKSTART_INSTALL.md [1]

---

## . The idea in one sentence

Start a prompt with a registered UPPERCASE token and that token is replaced by a full, pre-agreed instruction — so you type five characters instead of a paragraph, and the agent behaves the same way every time.

```
BUG: subtitles render one frame late on the secondary display
```

…is executed as *"REPORT DIRECTIVE — BUG (§11.4.202). The remainder of this prompt is a REPORT of a product DEFECT… create it through the project's reporting engine… never merely acknowledged, never silently dropped…"* followed by your actual text.

---

## . The five directives

| Directive | Reach for it when | What the agent then does |
|---|---|---|
| `BUG` | Something is **broken** — a regression, a defect, user-visible wrong behaviour. | Creates a fully-populated **Type=Bug** workable item (Status `Queued`, stable ticket id, structured description), regenerates every derived document from the DB, pushes to every configured external tracker, reports the ticket id back — then, if actionable now, investigates it under systematic-debugging (root cause **before** any fix). |
| `TASK` | Internal **workstream** — a refactor, a doc, infra, a gate, an audit, a chore. | Same pipeline, **Type=Task** (fixed). |
| `ISSUE` | **Anything trackable** and you have not decided what it is. | Classifies it into the closed set `{Bug \| Feature \| Task}` from the report's own content. If the content does not determine the type it **asks you** before creating anything. Then the same create + sync + push pipeline. |
| `BACKGROUND` | Work that should proceed **in parallel with the main stream** and must never be lost. | Records the request in the durable background queue immediately (id + timestamp + status + blocker), then runs it subagent-driven in parallel. If it cannot run now it stays **QUEUED** with its blocker recorded and is re-attempted at the next opportunity — including in a fresh session. It never interrupts the main stream, and it is never silently dropped. |
| `REMINDER` | You are re-surfacing earlier work whose **status you are unsure of**. | Does **not** assume it is done or not done. First verifies the actual status from captured evidence (git state, the task list, the background queue, test artifacts, the workable-items DB). Then acts on the delta: report the proof if genuinely complete, resume from the exact point if partial, start it now if not started, or surface the block. It always produces a status verdict. |

`ISSUE` **is a channel, not a type.** There is no "Issue" type — every item is a `Bug`, a `Feature`, or a `Task`.[3]

---

## . How to invoke — six equivalent forms

All six do exactly the same thing. Use whichever you like.

```
BUG: subtitles are one frame late              ← 6. single-colon (what
most people type)
BUG :: subtitles are one frame late            ← 1. bare  ::
DEFAULT::BUG :: subtitles are one frame late   ← 2. namespaced ::
BUG ---> subtitles are one frame late          ← 5. arrow
/DEFAULT::BUG subtitles are one frame late     ← 4. namespaced slash
/bug subtitles are one frame late              ← 3. bare slash — SEE §5, /
bug COLLIDES
```

The rules, in short:

- **Only the first non-blank line counts.** A token in the middle of a paragraph never triggers.
- **UPPERCASE only.** `bug: …` is ordinary text.
- **Spacing is exact.** `::` and `--->` need one space on each side; `NAME:` needs the colon tight against the name and one space after it. This is what keeps `std::vector`, YAML `key: value`, and URLs from ever triggering.
- **You can stack them.** `BACKGROUND :: BUG: the HDMI sink drops to stereo` applies the outer directive, then the inner one, then the task.

### Worked examples

```
TASK: bump the pandoc pin to 3.2 and regenerate the exports

ISSUE: playback stutters on 4K HDR after ~40 minutes — happens on both
boxes

BACKGROUND ---> audit every gate for a paired mutation and file what is
missing

REMINDER: the OTA signing-key rotation we scheduled last week
```

4

---

## . Escaping — talking about a directive without firing it

Put a single backslash in front. The backslash is stripped and nothing expands:

```
\BUG: this is an example of the syntax, do not open a ticket
\/BACKGROUND likewise
```

5

---

## . The collision: `/bug` vs `/helix:bug` vs `/default-bug`

Claude Code ships its **own built-in** `/bug` (report a bug to Anthropic). The registry declares that collision as data ( `slash_conflicts: [bug]` ), and the system respects it — **the plugin never silently shadows a host command.**

| You type | What runs |
| --- | --- |
| `/bug audio is silent` | **Claude Code's built-in.** Not the directive. |
| `/helix:bug audio is silent` | **The directive.** Plugin namespacing is always unambiguous. |
| `/default-bug audio is silent` | **The directive.** The collision-free alias — works on every agent. |
| `BUG: audio is silent` | **The directive.** No host command can touch this form. |
| `BUG :: audio is silent` / `BUG ---> audio is silent` | **The directive.** |

So: for `BUG`, use `BUG:` (simplest) or `/helix:bug` / `/default-bug`.

The other four directives have **no** declared collision, so their bare slash forms work directly: `/task …`, `/issue …`, `/background …`, `/reminder …` — and `/helix:task` / `/default-task` remain available if your host ever adds a colliding command.
[6]

---

## . What happens if you type a token that is not registered

The system **never invents an expansion** and **never silently drops your prompt**.

| You type | What happens |
| --- | --- |
| `BUGG :: audio is silent` (typo, `::` form) | You are **asked** which registered directive you meant — with the closest name suggested. |
| `/deploy the new image` (unregistered slash) | Same: asked, never guessed. |
| `NOTE: remember to re-flash D4` (unregistered **single-colon**) | **Nothing happens** — it is treated as an ordinary sentence. This is deliberate: `NOTE:` / `TODO:` / `WARNING:` / `FIXME:` are normal English, and questioning every one of them would be intolerable. |

7

---

## . Sub-system shortcuts

An UPPERCASE token that is not a directive but *is* an incorporated sub-system or submodule expands to that sub-system's working context (repository, org, where it is checked out, plus the rules for working on it — treat it as an equal part of the codebase, keep it decoupled, fetch first, never force-push):

```
HXQA :: add a Challenge bank entry for the subtitle oracle
HELIXOTA ---> investigate the delta-update failure on the 1.2.1 image
```

Tokens come from the curated catalogue ( `HXOTA` , `HXQA` , `HXCODE` , `HXAGENT` , `HXLLM` , `HXMEM` , `HXSPEC` , `HXTRACK` , `LLMSVERIFIER` , `CTOOLKIT` , …) **and** from your own project's submodules, discovered automatically from `.gitmodules` — a newly-added submodule works with no configuration.

If a token is ambiguous (two sub-systems), it is **dropped** and you are asked. A registered directive always wins over a sub-system of the same name.

---

## . How to see what is registered

```
# every registered action, one per line
bash -c '. constitution/scripts/action_prefix_lib.sh && apx_list_actions'

# what one directive actually expands to
bash -c '. constitution/scripts/action_prefix_lib.sh && apx_lookup_expansion BUG'

# dry-run the whole expander on a prompt (JSON: verdict / action / expansion / emitted)
bash -c '. constitution/scripts/action_prefix_lib.sh && apx_expand_prompt "BUG: audio is silent"'
```

Or just read the source of truth — `constitution/actions/registry.yaml` — and the generated command inventory, `constitution/plugins/helix/README.md`.

In Claude Code, `/helix:` then Tab lists every command the plugin exposes.[9]

---

## . If a directive does not seem to fire

1. Is it on the **first non-blank line**? A directive in the middle of a prompt never fires.
2. Is it **UPPERCASE**?
3. Is the **spacing exact**? `::` (spaces both sides), `NAME:` (no space before the colon, one after), `--->` (spaces both sides).
4. Did you use a **bare `/name` that collides** with a host command ( `/bug` )? Use `/helix:bug` or `/default-bug` (§5).
5. Still nothing? The wiring may not be installed in this project — hand ADMIN_MANUAL.md to whoever set it up, or run the one-line installer in QUICKSTART_INSTALL.md.